PROBLEMA:	Introdução ao Diagrama de Classes	DATA:	14/08/2026
PILOTO (A):	Jadson da Silva Sobrinho	COPILOTO (A):	Levi Falcão de Queiroz
QA:	Mateus Queiroz Matos Ferreira	ARQUITETO (A):	Moisés de Souza Olivera
SCRUM MASTER:	Murillo dos Santos Marinho Ferreira		

Objetivo: Modelar as entidades de negócio essenciais e iniciar o Diagrama de Classes com foco nos conceitos iniciais de Orientação a Objetos.

Fatos:

(O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT 1 e o Arquiteto documenta aqui no caderno):

1. **Atributos de Peça:** Analisando a seção "Modelagem dos Dados" da OAT, quais são os atributos obrigatórios que devem ser implementados na entidade Peça e quais são estritamente opcionais?

   Fato Extraído:
   Obrigatórios: Código único, nome, código de barras, fornecedor/marca, quantidade em estoque, preço de custo, preço de venda, data de cadastro e data da última atualização.
   Opcionais: Tamanho e cor.

2. **Atributos de Serviço:** De acordo com a especificação do produto na OAT, quais são os dados exatos que a entidade Serviço deve possuir para ser cadastrada validamente?

   Fato Extraído:

   Obrigatórios: Deve possuir um código único gerado pela aplicação, nome

do serviço, tempo estimado de duração em minutos, custo tabelado, data de criação e data de última atualização.

3. **A Obrigação do Enum:** O que a diretoria determina na especificação sobre a organização do catálogo utilizando Enumerações? Qual deve ser o nome desse Enum e como ele se vincula à entidade Peça?

   Fato Extraído:

   - Criar um enum chamado “CategoriaPeca”.
   - Vincular como um atributo obrigatório na entidade Peça.
   - MOTOR, SUSPENSAO, FREIOS, ELETRICA, ACESSORIOS

Brainstorm Investigativo:

(Questões e Ideias guiaTdas pelo Scrum Master)

1. A Modelagem Inicial: Como aplicar o encapsulamento nessas entidades para garantir a integridade dos dados? Quais modificadores de acesso usaremos para os atributos e métodos?

   Modificador de acesso vai ser Private

   PEÇA:

   Código único: Long

   nome da peça: String

   código de barras: String

   fornecedor/marca: String

   quantidade em estoque: Int

   preço de custo: Double

   preço de venda: Double

   data de cadastro: LocalDateTime

data da última atualização: LocalDateTime

Modificador de acesso vai ser Private

SERVIÇOS:

Código de único: Long

nome do serviço: String

tempo estimado (duração em minutos): String

custo tabelado: Double

data de criação: LocalDateTime

data de última atualização: LocalDateTime

2. O Construtor: Quais atributos devem ser obrigatórios no momento da instanciação (no construtor) e quais atributos dependerão puramente de métodos de acesso (Getters/Setters)?

   Todos os atributos deverão ser

Cenários de Teste:

(Liderado pelo QA)

- Validar se a criação de um objeto Peça exige as categorias restritas definidas no Enum CategoriaPeca.

ID do Teste: [Ex: QA-01]
Responsável (QA): [Nome de quem escreveu o cenário]
O que estamos testando? (Contexto): Se o usuário informou uma categoria que existe no enum CategoriaPeca
Cenário (Ação e Resultado):

- **Dado que** o usuário passou a Categoria da Peça como MOTOR
- **Quando** ele enviou o cadastro da peçå
- **Então** o sistema deve validar se a categoria enviada existe no enum Categoria e informar se ele existe ou não.

Status: [⏳ Pendente | ✅ Passou | ❌ Falhou]
Observações do Piloto/Dev: [Espaço para justificar caso tenha falhado]

Entrega do Encontro: Código: Criação de Peca.java, Servico.java e do Enum CategoriaPeca.java no repositório.

- Diagrama: O Arquiteto deve iniciar o Diagrama de Classes com as entidades e o Enum preenchidos com atributos e modificadores de visibilidade.

PROBLEMA:	Introdução ao Diagrama de Classes	DATA:	14/08/2026
PILOTO (A):		COPILOTO (A):	
QA:		ARQUITETO (A):	
SCRUM MASTER:			

Objetivo: Dominar a aplicação prática de membros estáticos (static) implementando a camada de acesso a dados (Repositórios) com o Padrão de Projeto *Singleton*.

Fatos:

(O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT 1 e o Arquiteto documenta no repositório)

1. **A Restrição Arquitetural:** Segundo a introdução da OAT e a seção "Motor de Armazenamento", quais são as tecnologias e recursos do Spring Boot (relacionados a banco de dados e injeção de dependência) que são estritamente proibidos nesta fase inicial?
    - **Fato Extraído:**
2. **O Padrão de Armazenamento:** A OAT exige a criação de repositórios manuais (ex: PecaRepository). Qual Padrão de Projeto deve ser obrigatoriamente utilizado para implementá-los e qual método deve conceder acesso exclusivo a eles?
    - **Fato Extraído:**

Brainstorm Investigativo:

(Questões e Ideias guiadas pelo Scrum Master)

- **O Construtor Privado:** Por que o construtor dessas classes repositório precisa ser *private*? O que isso impede que os programadores façam no futuro?

- **A Sobrevivência dos Dados:** Como as listas internas de persistência vão sobreviver e se manter as mesmas enquanto o servidor estiver rodando? Qual é o papel do atributo *static* nesse comportamento?

Cenários de Teste:

(Liderado pelo QA)

- Testar se o programa falha em tempo de compilação ao tentarmos instanciar o PecaRepository usando a palavra-chave new fora da própria classe.

Entrega do Encontro:

- **Código:** As classes PecaRepository.java e ServicoRepository.java implementando o padrão Singleton de forma manual, contendo as listas internas.
- **Diagrama:** Atualizar o Diagrama de Classes, evidenciando o sublinhado em membros UML que indica que os atributos e métodos getInstance() são estáticos.

PROBLEMA:	Introdução ao Diagrama de Classes	DATA:	28/08/2026
PILOTO (A):		COPILOTO (A):	
QA:		ARQUITETO (A):	
SCRUM MASTER:			

Objetivo: Mapear rotas na Web API estilo REST, compreender as anotações do framework Spring Boot e estabelecer os relacionamentos lógicos entre a interface web e os repositórios.

Fatos:

(O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT 1 e o Arquiteto documenta no repositório)

1. **Rotas e Controladores:** De acordo com a seção "Roteamento REST e Controladores" da OAT, qual é o padrão estrutural que as rotas da nossa aplicação devem seguir para expor os dados das peças e serviços?
    - **Fato Extraído:**
2. **A Semântica de Sucesso:** A OAT determina que a API não pode simplesmente devolver os dados puros. Quais são os Status HTTP exatos que devem ser retornados para Criação com sucesso, Listagens/Atualizações, e Deleções bem sucedidas?
    - **Fato Extraído:**
3. **A Semântica de Erro:** Qual Status HTTP a OAT exige que seja codificado e retornado caso uma operação de busca ou deleção não encontre o ID solicitado pelo cliente?
    - **Fato Extraído:**

Brainstorm Investigativo:

(Questões e Ideias guiadas pelo Scrum Master)

- **Integração:** Como faremos com que o Controller peça para o Repository salvar o dado, lembrando que a OAT proíbe a injeção de dependência padrão do Spring?
- **Relacionamentos no Diagrama:** Como representaremos no Diagrama de Classes a relação visual entre o PecaController e o PecaRepository?

Cenários de Teste:

(Liderado pelo QA)

- Validar a execução de uma requisição POST que deve resultar na gravação em memória e em um retorno 201 Created.

- Validar o retorno manual de 404 Not Found ao tentar buscar uma Peça cujo ID ainda não foi cadastrado no Singleton.

Entrega do Encontro:

- **Código:** PecaController.java e ServicoController.java com endpoints implementados para suprir as 8 User Stories.
- **Artefatos:** *Collection* (Postman ou Insomnia) exportada com as rotas criadas. O Arquiteto finaliza o Diagrama de Classes.